

What I Found Valuable So Far in CS6650

Thinking in Trade-offs

The biggest mindset shift has been evaluating every design decision as a trade-off. CAP theorem was a turning point — you can't have consistency, availability, and partition tolerance all at once, so you must decide what matters most per use case. This applies everywhere: TCP vs UDP, strong vs eventual consistency, and in HW6 — vertical scaling (simple but limited) vs horizontal scaling (complex but fault tolerant).

The 8 Fallacies and Designing for Failure

The 8 Fallacies of Distributed Computing from the first reading stuck with me: the network is not reliable, latency is not zero. Every assignment reinforced this — from handling timeouts in HW1's load tests to HW6's resilience test where stopping an instance caused zero failures because the ALB automatically routed around it.

REST APIs and gRPC

HW1 and HW5 gave me hands-on experience building REST APIs in Go — designing resource-oriented endpoints, using HTTP methods (GET/POST), and validating inputs. The readings also introduced gRPC as an alternative: using Protocol Buffers for binary serialization and HTTP/2 for transport, making it faster than REST for internal microservice communication. The trade-off is clear — REST is human-readable and great for public APIs, while gRPC is optimized for performance between services.

Microservices and Information Hiding

Parnas's 1972 paper on modular decomposition maps directly to modern microservices. Each service hides its internal decisions and exposes only an API. This helped me appreciate why the MapReduce pattern in HW4 works — splitting responsibilities into independent mappers and reducers that communicate through defined interfaces.

Concurrency and Race Conditions

Building multi-user services taught me why concurrency control matters. In HW5, multiple users could read and write to the same product data simultaneously, so I used a read-write mutex (`sync.RWMutex`) to prevent race conditions — allowing concurrent reads but exclusive writes. In HW6, `sync.Map` provided built-in thread safety. These are the same problems that appear at larger scale in distributed systems with replicated data, just at a different level.

From Single Server to Distributed Systems

The progression across assignments was well designed. HW1 started with a single Go server on EC2. HW4 introduced distributed processing with MapReduce across multiple containers. HW5 added load testing with Locust, comparing `HttpUser` vs `FastHttpUser`. HW6 brought it all together: identifying CPU as the bottleneck on a single instance (28% → 85%), then solving it with horizontal scaling using ALB, Auto Scaling, and multiple ECS Fargate tasks.

AWS Tooling and Infrastructure as Code

The assignments gave me experience with the full cloud stack: EC2, ECR, ECS Fargate, ALB, CloudWatch, and Auto Scaling. Defining everything with Terraform was particularly valuable — deploying and destroying a complete distributed system with one command. Learning to read CloudWatch metrics to make data-driven scaling decisions felt like a practical industry skill.

Caching and Consistency

The readings introduced caching vs replication as different strategies — caching keeps potentially stale local copies, while replication actively syncs. Understanding the spectrum from strong consistency (linearizable) to weak (eventual) helped me realize that the right consistency level depends entirely on the use case.

